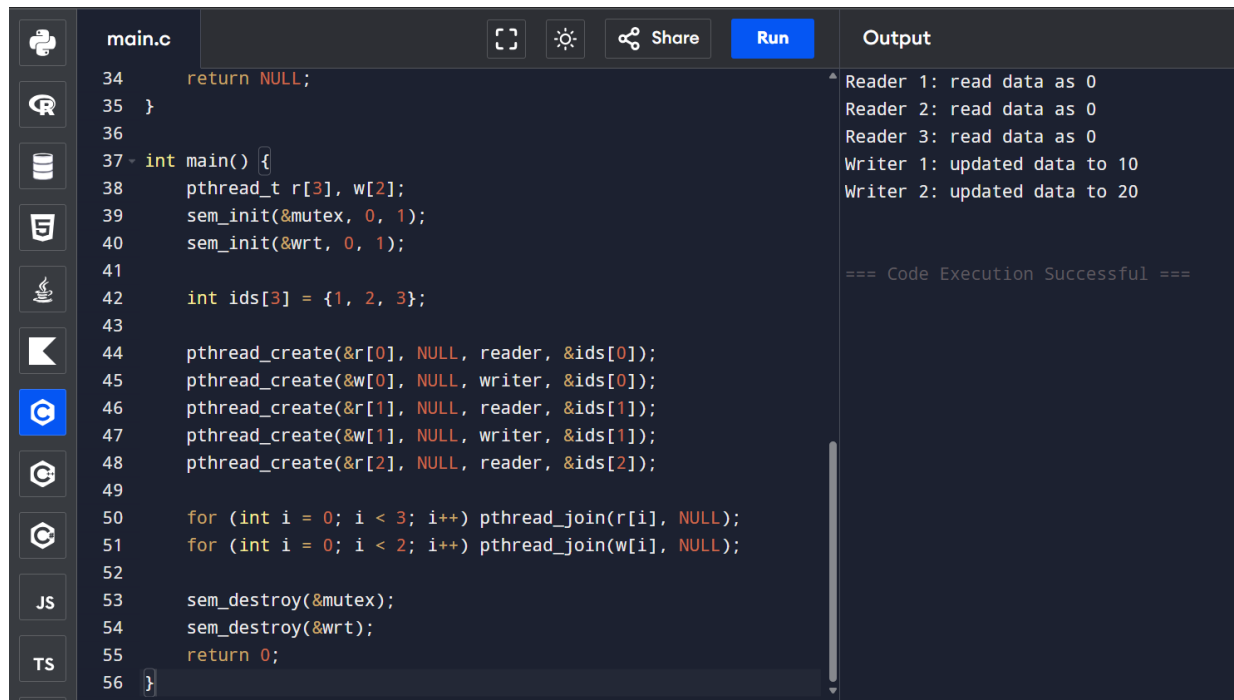


## EXPERIMENT 20:

20. Construct a C program to simulate Reader-Writer problem using Semaphores.



The screenshot shows a code editor with a C program named `main.c`. The code implements a Reader-Writer problem using semaphores. It defines two semaphores, `mutex` and `wrt`, both initialized to 1. Three reader threads and two writer threads are created, each with a unique ID. The reader threads read data, and the writer threads update data. The program uses `pthread_create` to spawn threads and `pthread_join` to wait for them to complete. The output shows the sequence of reads and updates, demonstrating that readers can read simultaneously, but writers must wait for all readers to finish before updating the data.

```
main.c
34     return NULL;
35 }
36
37 int main() {
38     pthread_t r[3], w[2];
39     sem_init(&mutex, 0, 1);
40     sem_init(&wrt, 0, 1);
41
42     int ids[3] = {1, 2, 3};
43
44     pthread_create(&r[0], NULL, reader, &ids[0]);
45     pthread_create(&w[0], NULL, writer, &ids[0]);
46     pthread_create(&r[1], NULL, reader, &ids[1]);
47     pthread_create(&w[1], NULL, writer, &ids[1]);
48     pthread_create(&r[2], NULL, reader, &ids[2]);
49
50     for (int i = 0; i < 3; i++) pthread_join(r[i], NULL);
51     for (int i = 0; i < 2; i++) pthread_join(w[i], NULL);
52
53     sem_destroy(&mutex);
54     sem_destroy(&wrt);
55     return 0;
56 }
```

Output

```
Reader 1: read data as 0
Reader 2: read data as 0
Reader 3: read data as 0
Writer 1: updated data to 10
Writer 2: updated data to 20

=== Code Execution Successful ===
```

## PSEUDO CODE :

Global Variables:

readCount = 0

Semaphore mutex = 1, wrt = 1

Reader Thread:

wait(mutex)

readCount++

if readCount == 1: wait(wrt) // First reader locks writers out

signal(mutex)

// Reading performed here

wait(mutex)

```
readCount--  
if readCount == 0: signal(wrt) // Last reader allows writers in  
signal(mutex)
```

Writer Thread:

```
wait(wrt)           // Claim exclusive write lock  
// Writing performed here  
signal(wrt)         // Release write lock
```

## CODE:

```
#include <stdio.h>  
  
#include <pthread.h>  
  
#include <semaphore.h>  
  
#include <unistd.h>
```

```
sem_t mutex, wrt;
```

```
int readCount = 0;
```

```
int data = 0;
```

```
void* reader(void* rNo) {
```

```
    sem_wait(&mutex);
```

```
    readCount++;
```

```
    if (readCount == 1) sem_wait(&wrt);
```

```
    sem_post(&mutex);
```

```
    printf("Reader %d: read data as %d\n", *((int*)rNo), data);
```

```
sleep(1);
```

```
sem_wait(&mutex);
```

```
readCount--;
```

```
if (readCount == 0) sem_post(&wrt);
```

```
sem_post(&mutex);
```

```
return NULL;
```

```
}
```

```
void* writer(void* wNo) {
```

```
    sem_wait(&wrt);
```

```
    data += 10;
```

```
    printf("Writer %d: updated data to %d\n", *((int*)wNo), data);
```

```
    sleep(1);
```

```
    sem_post(&wrt);
```

```
    return NULL;
```

```
}
```

```
int main() {
```

```
    pthread_t r[3], w[2];
```

```
    sem_init(&mutex, 0, 1);
```

```
    sem_init(&wrt, 0, 1);
```

```
int ids[3] = {1, 2, 3};
```

```
pthread_create(&r[0], NULL, reader, &ids[0]);
```

```
pthread_create(&w[0], NULL, writer, &ids[0]);
```

```
pthread_create(&r[1], NULL, reader, &ids[1]);
```

```
pthread_create(&w[1], NULL, writer, &ids[1]);
```

```
pthread_create(&r[2], NULL, reader, &ids[2]);
```

```
for (int i = 0; i < 3; i++) pthread_join(r[i], NULL);
```

```
for (int i = 0; i < 2; i++) pthread_join(w[i], NULL);
```

```
sem_destroy(&mutex);
```

```
sem_destroy(&wrt);
```

```
return 0;
```

```
}
```